

Examen transversal: Comunicaciones unificadas

Nombre: Felipe Hodges Salgado

Profesor: Sebastián Pizarro

Fecha: 15-07-2026

Contenido

Introducción	3
Desarrollo	5
Conclusión	19

Introducción

Contexto:

La Clínica Regional "Salud Integral" es un centro médico de mediano tamaño que atiende a más de 15,000 pacientes al año. Actualmente enfrentan una problemática crítica: el 30% de las citas programadas resultan en ausencias de pacientes (no-show), generando pérdidas económicas significativas y desperdicio de tiempo médico especializado. El equipo de TI de la clínica ha identificado que la principal causa es la falta de recordatorios efectivos. Actualmente dependen de llamadas manuales realizadas por personal administrativo, proceso que es:

- Ineficiente (solo se logra contactar al 40% de los pacientes)
- Costoso (consume 15 horas/semana de personal)
- Propenso a errores humanos
- Sin registro automatizado de confirmaciones

Situación Inicial (Problemática):

La clínica cuenta con:

- Sistema de gestión de citas (base de datos local)
- Infraestructura tecnológica limitada
- Personal administrativo sobrecargado
- Sin sistema de telecomunicaciones VoIP
- Preocupaciones sobre privacidad de datos médicos (sensibles)

Requerimientos del Proyecto:

La dirección de la clínica solicita implementar un Sistema Automatizado de Confirmación de Citas que:

1. Realice llamadas automáticas a pacientes 24-48 horas antes de su cita
2. Proporcione información personalizada (nombre, especialidad, fecha, hora)
3. Capture la respuesta del paciente mediante teclado telefónico:
 - o 1 = Confirmar asistencia
 - o 2 = Cancelar cita

o 3 = Solicitar reprogramación

4. Actualice automáticamente el sistema de gestión de citas.
5. Genere reportes de confirmaciones/cancelaciones.
6. Garantice seguridad y privacidad de la información médica.
7. Sea escalable para el futuro crecimiento de la clínica.

Restricciones y Consideraciones:

- Presupuesto limitado: Priorizar soluciones open-source y cloud gratuito.
- Cumplimiento normativo: Los datos de pacientes son sensibles (similar a GDPR/Ley 19.628).
- Disponibilidad: El sistema debe operar 24/7.
- Usabilidad: Interfaz simple para personal administrativo no técnico.
- Seguridad: Cifrado obligatorio para proteger datos médicos en tránsito.

Desarrollo

1. Implementación de Central Telefónica:

Se desplegó una central una VM con Asterisk en la nube AWS

▼ Resumen

Número de instancias

Información

1

Imagen de software (AMI)

Canonical, Ubuntu, 24.04, amd64...[más información](#)

ami-0f8a61b66d1accaee

Tipo de servidor virtual (tipo de instancia)

t3.medium

Firewall (grupo de seguridad)

Nuevo grupo de seguridad

Almacenamiento (volúmenes)

Volúmenes: 1 (30 GiB)

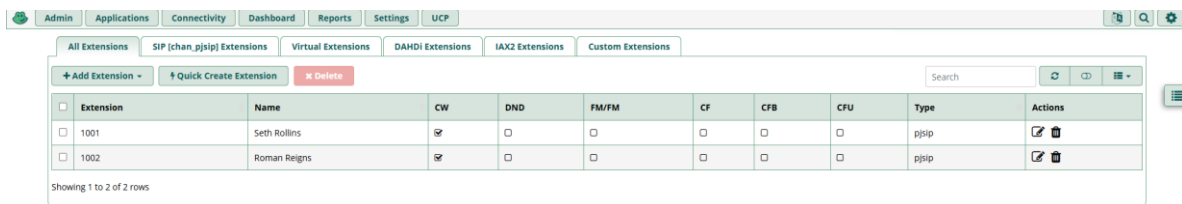
Cancelar

Lanzar instancia

 Código de versión preliminar

Como primer paso se crearon dos extensiones PJSIP en FreePBX (1001 y 1002), las cuales fueron utilizadas para simular la comunicación entre pacientes y la central telefónica.

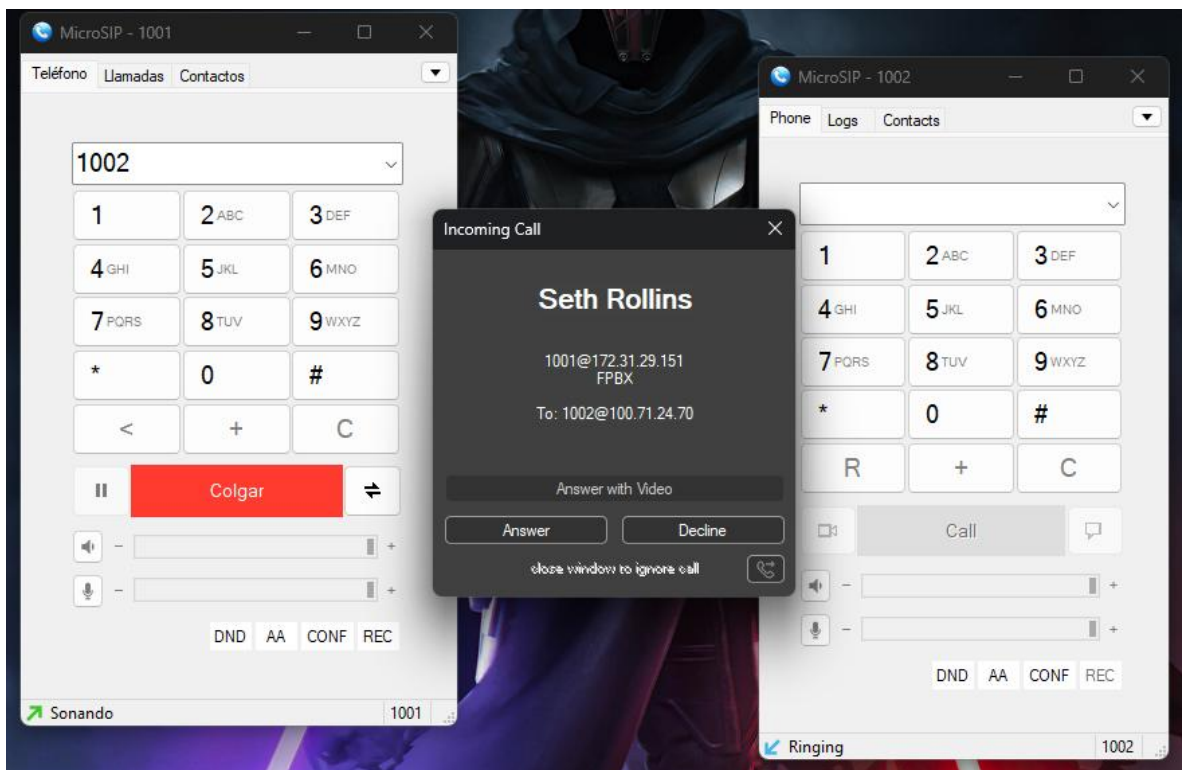
Posteriormente ambas extensiones fueron registradas correctamente mediante el softphone MicroSIP, verificando que pudieran realizar llamadas internas sin inconvenientes.



Extension	Name	CW	DND	FM/FM	CF	CFB	CFU	Type	Actions
1001	Seth Rollins	☒	☐	☐	☐	☐	☐	pjsip	
1002	Roman Reigns	☒	☐	☐	☐	☐	☐	pjsip	

Showing 1 to 2 of 2 rows

Se verificó el correcto registro de ambas extensiones SIP en la PBX. Posteriormente se realizó una llamada entre las extensiones 1001 y 1002 utilizando MicroSIP como cliente SIP, validando el establecimiento de la sesión mediante el protocolo PJSIP.



Se comprobó el correcto funcionamiento de las extensiones utilizando la consola de Asterisk mediante el comando `pjsip show endpoints`.

```

ubuntu@ip-172-31-29-151: ~
$
This is free software, with components licensed under the GNU General Public
License version 2 and other licenses; you are welcome to redistribute it under
certain conditions. Type 'core show license' for details.
=====
Running as user 'asterisk'
Running under group 'asterisk'
Connected to Asterisk 22.10.1 currently running on ip-172-31-29-151 (pid = 42161
)
ip-172-31-29-151*CLI> pjsip show endpoints

Endpoint: <Endpoint/CID.....> <State.....> <Channels.>
  I/OAuth: <AuthId/UserName.....>
  Aor: <Aor.....> <MaxContact>
  Contact: <Aor/ContactUri.....> <Hash.....> <Status> <RTT(ms) ..>
  Transport: <TransportId.....> <Type> <cos> <tos> <BindAddress.....>
  Identify: <Identify/Endpoint.....>
  Match: <criteria.....>
  Channel: <ChannelId.....> <State.....> <Time.....>
  Exten: <DialedExten.....> CLCID: <ConnectedLineCID.....>
=====

Endpoint: 1001/1001                               Unavailable  0 of inf
  InAuth: 1001-auth/1001
  Aor: 1001                                         1

Endpoint: 1002/1002                               Unavailable  0 of inf
  InAuth: 1002-auth/1002
  Aor: 1002                                         1

Objects found: 2

```

Inicialmente las extensiones aparecían como **Unavailable**, debido a que aún no estaban registradas desde MicroSIP. Posteriormente, al registrar ambos teléfonos IP, las llamadas internas fueron exitosas.

Implementación script AGI

El sistema fue desarrollado utilizando Python mediante la interfaz AGI de Asterisk.

El script implementado realiza las siguientes funciones:

- Recepción automática de llamadas.
- Consulta de la información de la cita almacenada en un archivo CSV.
- Reproducción de mensajes personalizados.
- Captura de respuestas mediante tonos DTMF.
- Registro de la respuesta del usuario.
- Actualización automática del estado de la cita.
- Generación de un archivo de log con cada ejecución.

```
def capturar_respuesta(agi, log):
    """
    Reproduce el menú y captura UNA tecla DTMF válida (1, 2 o 3).
    Reintenta hasta 3 veces. Devuelve la tecla (str) o None.
    """
    MAX_INTENTOS = 3
    for intento in range(MAX_INTENTOS):
        # get_data reproduce el audio y captura dígitos.
        # timeout en ms, max_digits=1 -> una sola tecla.
        tecla = agi.get_data(AUDIO_MENU, 5000, 1)
        if tecla and tecla in DTMF_A_OPCION:
            return tecla
        log.info("Intento %d: tecla inválida o sin respuesta (%r)",
                 intento + 1, tecla)
    return None

def actualizar_estado_cita(anexo, nuevo_estado):
    """
    Actualiza el estado de la cita en citas.csv.
    """
    filas = []
    encontrado = False

    with open(CITAS_CSV, newline="", encoding="utf-8") as f:
        lector = csv.DictReader(f)
        campos = lector.fieldnames or []

        if "estado" not in campos:
            campos.append("estado")

        for fila in lector:
            if fila.get("anexo") == str(anexo):
                fila["estado"] = nuevo_estado
                encontrado = True
            elif not fila.get("estado"):
                fila["estado"] = "pendiente"

            filas.append(fila)

    if not encontrado:
        return False

    temporal = CITAS_CSV + ".tmp"

    with open(temporal, "w", newline="", encoding="utf-8") as f:
        escritor = csv.DictWriter(f, fieldnames=campos)
        escritor.writeheader()
        escritor.writerows(filas)

    os.replace(temporal, CITAS_CSV)
    return True

# =====
# MAIN
# =====
def main():
```

Para integrar el script AGI con Asterisk se modificó el archivo extensions_custom.conf

agregando un nuevo contexto denominado: confirmacion-citas

Además se creó la extensión **7000**, utilizada para ejecutar el sistema de confirmación desde cualquier teléfono registrado.

```

=====
; extensions_custom.conf -- Sistema de confirmación de citas (CUY5132)
=====
; Pegar el contenido de este archivo al final de:
; /etc/asterisk/extensions_custom.conf
; y recargar:
; sudo asterisk -rx "dialplan reload"
;
; IMPORTANTE: en FreePBX el dialplan personalizado va SIEMPRE en
; extensions_custom.conf. Nunca editen extensions.conf (FreePBX lo regenera).
=====

; --- Contexto del sistema de confirmación ---
[confirmacion-citas]
exten => _10XX,1,NoOp(=== Inicio confirmacion de cita para ${EXTEN} ===)
same => n,Answer()
same => n,Wait(1)
same => n,AGI(confirmar_cita.agi,${EXTEN})
same => n,Hangup()

exten => s,1,NoOp(=== Confirmacion de cita - entrada generica ===)
same => n,Answer()
same => n,Wait(1)
same => n,AGI(confirmar_cita.agi,${CALLERID(num)})
same => n,Hangup()

; --- Punto de entrada de PRUEBA (marcable desde un softphone) ---
; Marca 7000 para probar el sistema simulando la cita del anexo 1001.
; Cambien el 1001 por el anexo que quieran probar (debe existir en citas.csv).
[from-internal-custom]
exten => 7000,1,NoOp(=== Prueba sistema confirmacion citas ===)
same => n,Goto(confirmacion-citas,1001,1)

```

La información de las citas fue almacenada en un archivo CSV.

Cada registro contiene, anexo, paciente, especialidad, fecha, hora, audio de la especialidad, estado

Durante las pruebas el sistema actualizó automáticamente el estado de la cita según la opción seleccionada por el usuario.

	A	B	C	D	E	F	G
1	anexo	paciente	especialidad	fecha	hora	audio_especialidad	estado
2	1001	Juan Perez	Cardiologia	2025-12-15	9:30	custom/esp_cardiolo	reprogramacion_sol
3	1002	Maria Gonzalez	Dermatologia	2025-12-15	11:00	custom/esp_dermato	pendiente
4	1003	Pedro Soto	Traumatologia	2025-12-16	8:15	custom/esp_traumat	pendiente

Finalmente se realizaron pruebas utilizando MicroSIP, marcando la extensión **7000**, comprobando que el sistema:

- Atiende automáticamente la llamada.
- Consulta correctamente la información de la cita.
- Reproduce los mensajes personalizados.
- Captura las opciones DTMF.
- Actualiza el estado de la cita.
- Registra la respuesta.

Los resultados obtenidos demostraron el correcto funcionamiento del sistema desarrollado.

```
Running as user 'asterisk'
Running under group 'asterisk'
Connected to Asterisk 22.10.1 currently running on ip-172-31-29-151 (pid = 42161)
== Using SIP RTP Audio TOS bits 184
== Using SIP RTP Audio TOS bits 184 in TCLASS field.
== Using SIP RTP Audio CoS mark 5
-- Executing [7000@from-internal:1] NoOp("PJSIP/1001-00000003", "=== Prueba sistema confirmacion citas ===") in new stack
-- Executing [7000@from-internal:2] Goto("PJSIP/1001-00000003", "confirmacion-citas,1001,1") in new stack
-- Goto (confirmacion-citas,1001,1)
-- Executing [1001@confirmacion-citas:1] NoOp("PJSIP/1001-00000003", "=== Inicio confirmacion de cita para 1001 ===") in new stack
-- Executing [1001@confirmacion-citas:2] Answer("PJSIP/1001-00000003", "") in new stack
-- Executing [1001@confirmacion-citas:3] Wait("PJSIP/1001-00000003", "1") in new stack
-- Executing [1001@confirmacion-citas:4] AGI("PJSIP/1001-00000003", "confirmar_cita.agi,1001") in new stack
-- Launched AGI Script /var/lib/asterisk/agi-bin/confirmar_cita.agi
confirmar_cita.agi,1001: Procesando confirmacion para anexo: 1001
-- <PJSIP/1001-00000003> Playing 'custom/bienvenida.slin' (escape_digits=) (sample_offset 0) (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/day-1.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/mon-11.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/15.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/2.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/thousand.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/20.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/5.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'custom/a_las.slin' (escape_digits=) (sample_offset 0) (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/9.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/30.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'digits/a-m.ulaw' (language 'en')
-- <PJSIP/1001-00000003> Playing 'custom/de.slin' (escape_digits=) (sample_offset 0) (language 'en')
-- <PJSIP/1001-00000003> Playing 'custom/esp cardiologia.slin' (escape_digits=) (sample_offset 0) (language 'en')
-- <PJSIP/1001-00000003> Playing 'custom/menu opciones.slin' (language 'en')
-- <PJSIP/1001-00000003> Playing 'custom/despida.slin' (escape_digits=) (sample_offset 0) (language 'en')
-- <PJSIP/1001-00000003> AGI Script confirmar_cita.agi completed, returning 4
== Spawn extension (confirmacion-citas, 1001, 4) exited non-zero on 'PJSIP/1001-00000003'
ip-172-31-29-151*CLI>
```

```
ubuntu@ip-172-31-29-151:~/NEW-CUY5132-DUOC/eft-confirmacion-citas$
```

2. Aseguramiento de la Plataforma

Con el objetivo de aumentar la seguridad de la solución VoIP, se implementó un Session Border Controller (SBC) utilizando Kamailio sobre una instancia independiente en Amazon Web Services (AWS EC2). Esta arquitectura permite desacoplar el acceso público de la PBX, evitando que los clientes SIP se conecten directamente al servidor Asterisk.

La nueva instancia fue creada con sistema operativo Ubuntu Server y configurada dentro de la misma VPC que la PBX, permitiendo la comunicación mediante direcciones IP privadas.

1 _____

i-01cd6d36646489cb3 (SBC kamailio)

[Detalles](#)
[Estado y alarmas](#)
[Monitoreo](#)
[Seguridad](#)
[Redes](#)
[Almacenamiento](#)
[Etiquetas](#)

▼ Resumen de instancia Información

ID de la instancia
i-01cd6d36646489cb3

Dirección IPv4 pública
 54.80.130.201 | [dirección abierta](#)

Direcciones IPv4 privadas
172.31.24.222

Se instaló Kamailio utilizando los repositorios oficiales de Ubuntu mediante el gestor de paquetes APT.

Posteriormente se verificó la instalación mediante comandos

```
ubuntu@ip-172-31-24-222:~$ kamailio -V
version: kamailio 5.7.4 (x86_64/linux)
flags: USE_TCP, USE_TLS, USE_SCTP, TLS_HOOKS, USE_RAW_SOCKETS, DISABLE_NAGLE, USE_MCAST, DNS_IP_HACK, SHM_MMAP, PKG_MALLOC, MEM_JOIN_FREE, Q_MALLOC, F_MALLOC, TLSF_MALLOC, DBG_SR_MEMORY, USE_FUTEX, FAST_LOCK-ADAPTIVE_WAIT, USE_DNS_CACHE, USE_DNS_FAILOVER, USE_NAPTR, USE_DST_BLOCKLIST, HAVE_RESOLV_RES, TLS_PTHREAD_MUTEX_SHARED
ADAPTIVE_WAIT_LOOPS 1024, MAX_RECV_BUFFER_SIZE 262144, MAX_URI_SIZE 1024, BUF_SIZE 65535, DEFAULT PKG_SIZE 8MB
poll method support: poll, epoll_lt, epoll_et, sigio_rt, select.
ld: unknown
compiled with gcc 13.2.0
ubuntu@ip-172-31-24-222:~$

ubuntu@ip-172-31-24-222:~$ systemctl status kamailio
● kamailio.service - Kamailio - the Open Source SIP Server
   Loaded: loaded (/usr/lib/systemd/system/kamailio.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-07-15 08:05:27 UTC; 13min ago
     Docs: man:kamailio(8)
   Process: 6919 ExecStart=/usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f $CFGFILE -m $SHM_MEMORY -M $PKG_MEMORY (code=exited, status=0/SUCCESS)
   Main PID: 6923 (kamailio)
      Tasks: 13 (limit: 4520)
     Memory: 8.9M (peak: 9.5M)
        CPU: 621ms
    CGroup: /system.slice/kamailio.service
            └─6923 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6924 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6925 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6926 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6927 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6928 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6929 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6930 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6931 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6932 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6933 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6934 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8
            └─6935 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamailio.cfg -m 64 -M 8

Jul 15 08:05:27 ip-172-31-24-222 kamailio[6919]:      udp: ip-172-31-24-222.ec2.internal:5060
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6923]: INFO: rr [./outbound/api.h:53]: ob_load_api(): unable to import bind_ob - maybe module is not loaded
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6923]: INFO: rr [rr_mod.c:185]: mod_init(): outbound module not available
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6923]: INFO: <core> [main.c:3189]: main(): processes (at least): 13 - shm size: 67108864 - pkg size: 83584
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6923]: INFO: <core> [core/udp_server.c:163]: probe_max_receive_buffer(): SO_RCVBUF is initially 212992 for
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6923]: INFO: <core> [core/udp_server.c:233]: probe_max_receive_buffer(): SO_RCVBUF is finally 425984 on fd
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6924]: INFO: rtpengine [rtpengine.c:3040]: rtp_test(): rtpengine instance 'udpi127.0.0.1:2222' found, su
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6931]: INFO: ctl [io_listener.c:215]: io_listen_loop(): io_listen_loop: using epoll_lt to watch method (d
Jul 15 08:05:27 ip-172-31-24-222 /usr/sbin/kamailio[6933]: INFO: jsonrpc [jsonrpc_mock.c:471]: jsonrpc_dgram_process(): a new child 0/6933
Jul 15 08:05:27 ip-172-31-24-222 systemd[1]: Started kamailio.service - Kamailio - the Open Source SIP Server.
ubuntu@ip-172-31-24-222:~$
```

Con el objetivo de administrar correctamente el flujo RTP y permitir futuras implementaciones de NAT Traversal y seguridad multimedia, se instaló RTPengine.

Posteriormente se comprobó su funcionamiento verificando el estado del servicio y el puerto de control utilizado por Kamailio.

```
ubuntu@ip-172-31-24-222:~$ systemctl status rtpengine-daemon
● rtpengine-daemon.service - RTP/media Proxy Daemon
   Loaded: loaded (/usr/lib/systemd/system/rtpengine-daemon.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-07-15 07:14:09 UTC; 1h 5min ago
     Docs: man:rtpengine(8)
   Process: 6654 ExecStartPre=/usr/sbin/rtpengine-iptables-setup start (code=exited, status=0/SUCCESS)
   Main PID: 6661 (rtpengine)
      Tasks: 25 (limit: 4520)
     Memory: 8.4M (peak: 8.6M)
        CPU: 557ms
    CGroup: /system.slice/rtpengine-daemon.service
            └─6661 /usr/bin/rtpengine -f -E --no-log-timestamps --pidfile /run/rtpengine/rtpengine-daemon.pid --config-file /etc/rtpengine/rtpengine.conf

Jul 15 07:14:09 ip-172-31-24-222 systemd[1]: Starting rtpengine-daemon.service - RTP/media Proxy Daemon...
Jul 15 07:14:09 ip-172-31-24-222 rtpengine[6661]: INFO: [crypto] Generating new DTLS certificate
Jul 15 07:14:09 ip-172-31-24-222 rtpengine[6661]: INFO: [core] Startup complete, version 11.5.1.18-lubuntul.2
Jul 15 07:14:09 ip-172-31-24-222 systemd[1]: Started rtpengine-daemon.service - RTP/media Proxy Daemon.
Jul 15 07:48:58 ip-172-31-24-222 rtpengine[6661]: INFO: [control] Received command 'ping' from 127.0.0.1:51313
Jul 15 07:48:58 ip-172-31-24-222 rtpengine[6661]: INFO: [control] Replying to 'ping' from 127.0.0.1:51313 (elapsed time 0.000001 sec)
Jul 15 08:05:27 ip-172-31-24-222 rtpengine[6661]: INFO: [control] Received command 'ping' from 127.0.0.1:53779
Jul 15 08:05:27 ip-172-31-24-222 rtpengine[6661]: INFO: [control] Replying to 'ping' from 127.0.0.1:53779 (elapsed time 0.000000 sec)
ubuntu@ip-172-31-24-222:~$
```

```
ubuntu@ip-172-31-24-222:~$ ss -ltn | grep 2222
UNCONN 0      0          127.0.0.1:2222      0.0.0.0:*
ubuntu@ip-172-31-24-222:~$
```

Los resultados confirmaron que RTPengine quedó ejecutándose correctamente y escuchando en el puerto UDP 2222.

Se modificó el archivo principal de configuración kamailio.cfg

Durante esta etapa se realizaron las siguientes modificaciones:

Configuración de la IP pública del SBC mediante el parámetro advertise.

Configuración del puerto SIP 5060.

Integración con RTPengine.

Verificación de la sintaxis del archivo antes de reiniciar el servicio.

```
ubuntu@ip-172-31-24-222:~$ ss -ltn | grep 22222
UNCONN 0      0      127.0.0.1:22222      0.0.0.0:*
ubuntu@ip-172-31-24-222:~$ sudo kamailio -c -f /etc/kamailio/kamailio.cfg
0(6993) INFO: <core> [core/sctp_core.c:74]: sctp_core_check_support(): Sctp API not enabled - if you want to use it, load sctp module
Listening on
      udp: 172.31.24.222:5060 advertise 54.80.130.201:5060
Aliases:
      udp: ip-172-31-24-222.ec2.internal:5060
config file ok, exiting...
ubuntu@ip-172-31-24-222:~$
```

Con el propósito de proteger la infraestructura SIP frente a posibles ataques y evitar la exposición de información interna de la PBX, se habilitó el módulo Topology Hiding (topoh) de Kamailio.

Se cargó el módulo correspondiente y se configuró una clave de enmascaramiento para ocultar los encabezados SIP que podrían revelar la topología interna del sistema.

```
ubuntu@ip-172-31-24-222:~$ grep -n 'topoh.so\|mask_key' /etc/kamailio/kamailio.cfg
44:loadmodule "topoh.so"
64:modparam("topoh", "mask_key", "CUY5132-SBC-2026")
ubuntu@ip-172-31-24-222:~$
```

```
ubuntu@ip-172-31-24-222:~$ sudo kamailio -c -f /etc/kamailio/kamailio.cfg
0(7160) INFO: <core> [core/sctp_core.c:74]: sctp_core_check_support(): Sctp API not enabled - if you want to use it, load sctp module
Listening on
      udp: 172.31.24.222:5060 advertise 54.80.130.201:5060
Aliases:
      udp: ip-172-31-24-222.ec2.internal:5060
config file ok, exiting...
ubuntu@ip-172-31-24-222:~$
```

Modificación kamailio.cfg

```
ubuntu@ip-172-31-24-222: ~  
GNU nano 7.2 /etc/kamailio/kamailio.cfg *  
# Kamailio escucha en su IP privada y anuncia su IP pública  
listen=udp:172.31.24.222:5060 advertise 54.80.130.201:5060  
  
# IP privada de FreePBX/Asterisk  
#!define ASTERISK_IP "172.31.29.151"  
  
# IP pública del SBC  
#!define KAMAILIO_IP "54.80.130.201"  
  
##### Modules Section #####  
  
loadmodule "tm.so"  
loadmodule "sl.so"  
loadmodule "rr.so"  
loadmodule "pv.so"  
loadmodule "maxfwd.so"  
loadmodule "usrloc.so"  
loadmodule "registrars.so"  
loadmodule "textops.so"  
loadmodule "siputils.so"  
loadmodule "xlog.so"  
loadmodule "ctl.so"  
loadmodule "jsonrpcs.so"  
loadmodule "kex.so"  
loadmodule "nathelper.so"  
loadmodule "rtppengine.so"  
loadmodule "topoh.so"  
loadmodule "tls.so"  
  
##### Module Parameters #####  
|
```

```
GNU nano 7.2 /etc/kamailio/kamailio.cfg *
children=4

# Kamailio escucha en su IP privada y anuncia su IP pública
enable_tls=yes

listen=tls:172.31.24.222:5061 advertise 54.80.130.201:5061
listen=udp:172.31.24.222:5060 advertise 54.80.130.201:5060

# IP privada de FreePBX/Asterisk
#!define ASTERISK_IP "172.31.29.151"

# IP pública del SBC
#!define KAMAILIO_IP "54.80.130.201"

##### Modules Section #####

loadmodule "tm.so"
loadmodule "sl.so"
loadmodule "rr.so"
loadmodule "pv.so"
loadmodule "maxfwd.so"
loadmodule "usrloc.so"
loadmodule "registrar.so"
loadmodule "textops.so"
loadmodule "siputils.so"
loadmodule "xlog.so"
loadmodule "ctl.so"
loadmodule "jsonrpcs.so"
loadmodule "kex.so"
loadmodule "nathelper.so"
loadmodule "rtppengine.so"
loadmodule "topoh.so"
loadmodule "tls.so"

##### Module Parameters #####

modparam("ctl", "binrpc", "unix:/var/run/kamailio/kamailio_ctl")
modparam("jsonrpcs", "fifo_name", "/run/kamailio/kamailio_rpc.fifo")

modparam("rr", "enable_full_lr", 1)
modparam("rr", "append_fromtag", 1)

modparam("registrar", "method_filtering", 1)
modparam("registrar", "max_expires", 3600)
modparam("registrar", "gruu_enabled", 0)

modparam("usrloc", "db_mode", 0)

modparam("nathelper", "natping_interval", 30)
modparam("nathelper", "ping_nated_only", 0)

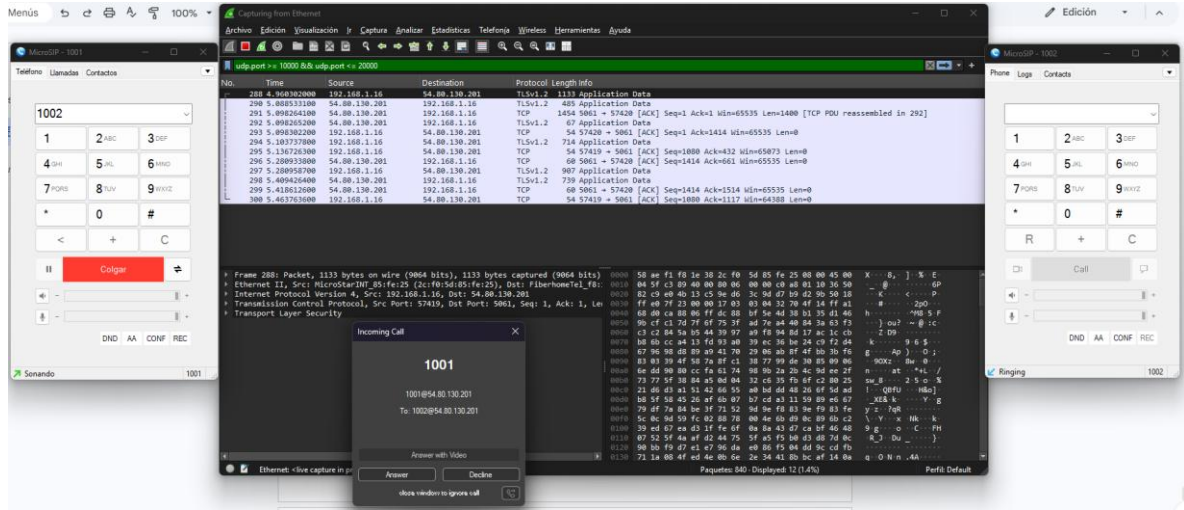
modparam("rtppengine", "rtppengine_sock", "udp:127.0.0.1:22222")
modparam("topoh", "mask_key", "CÚY5132-SBC-2026")
modparam("tls", "config", "/etc/kamailio/tls.cfg")

##### Routing Logic #####
```

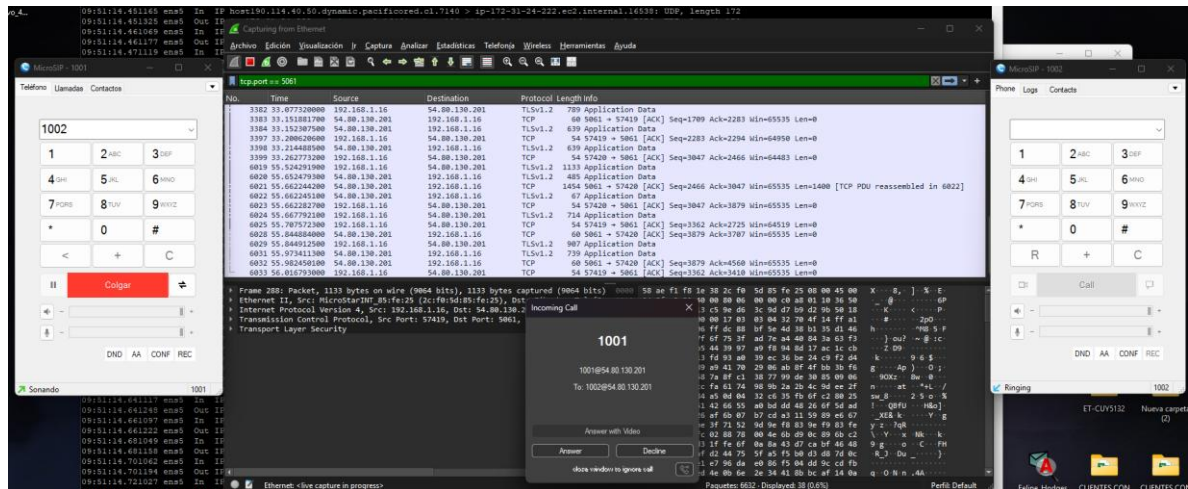
Se validó el funcionamiento del transporte SIP sobre TLS mediante openssl s_client, conectándose al puerto 5061 del SBC. La prueba confirmó el establecimiento exitoso del handshake TLS, la presentación del certificado autofirmado con Common Name correspondiente a la IP pública del SBC y la negociación de TLS 1.3 con cifrado AES-256-GCM. El aviso de certificado autofirmado es esperado en un entorno de laboratorio y no impide el cifrado de la señalización.

```
ubuntu@ip-172-31-24-222:/etc/kamailio/certs$ echo | openssl s_client -connect 172.31.24.222:5061 -servername 54.80.130.201
CONNECTED(00000003)
depth=0 C = CL, ST = RM, L = Santiago, O = DUOC-UC, OU = VoIP-Lab, CN = 54.80.130.201
verify error:num=18:self-signed certificate
verify return:1
depth=0 C = CL, ST = RM, L = Santiago, O = DUOC-UC, OU = VoIP-Lab, CN = 54.80.130.201
verify return:1
---
Certificate chain
 0 s:C = CL, ST = RM, L = Santiago, O = DUOC-UC, OU = VoIP-Lab, CN = 54.80.130.201
  i:C = CL, ST = RM, L = Santiago, O = DUOC-UC, OU = VoIP-Lab, CN = 54.80.130.201
  a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Jul 15 08:57:07 2026 GMT; NotAfter: Jul 15 08:57:07 2027 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIDtTCCAp2gAwIBAgIUOLrLSIDRVdKWdsk7XR1TJIrn2GowDQYJKoZIhvcNAQEL
BQAwajELMAkGA1UEBhMCQ0wxZzA5bGAgNVBAgMA1JNMREwDwYDVQOHDAhTYW50aWFn
bzEzEQA4GAlUECgwHRFVFPQy1VQzERMA8GAlUECwIVm9JUC1MYWlxFjAUBgNVBAMM
DTU0LjgwLjEzMC4yMDEwHhcNMjYwNzE1MDg1NzA3WmcNMjYwNzE1MDg1NzA3WjBq
MQswCQYDVQGEwJDTDELMkGA1UECAwCUk0xETAPBgNVBACMCFFhbnRyYXVdYWRMRWAw
DQYDVQQKDAkEVU9DLVVMREwDwYDVQQLDAhWb010LXhYjEwMBQGA1UEAwwNNNTQu
ODAuMTMwLjIwMTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAMub6j48
qaOxoBlvAF1Brn9BC/1wB1+03aUfbAg8yDnngt3pZD5wp94jO9afL0Hyi1DGmyY
uVjwgNkmS1wU5oc+rmYu0fhwV6QrBF0mv6GGKd69/T4FHZI8TFM2eIdRbQxfBPFR
ggf+aRdnxTqu6Czr3ASU8TXgFudb8bg+EHeu/NSFsGTM06NpgEBoBTbT4I1WgK4B
vY58K+yPCcCon221hODMNZKiUWYuCh0ehhh1U/D0ygIsUo1VBybAp4SAPP7ZwWtz
xYYEQX8NUAmlYt/n/DOT7FS5Lr6NPdZYktfAMW/2n9j91VGixPrDvAiaxnMPFK
CQgSPeIGKxNwOkCAwEAAANTMFwHQYDVR0OBBYEFMjYyX9frmdEX52E0rGS6+uLW
SgQQMB8GAlUdIwQYMBaAFMjYyX9frmdEX52E0rGS6+uLWSgQQMA8GAlUdEwEB/wQF
MAMBAf8wDQYJKoZIhvcNAQELBQADggEBAHirKREKnmfWwUMpEhwY7itHmT2MXf1
Fmm53esVD4Nou+XEzY6+LJy0/ILqDRTZbHohSv7vq2hV9PogUMITsvOdsVDy0xLK
Lb+sv3vapj2yuiXWXXanfwmtdQD7y0eOjPgYINfd721xJlw+2jrb9C6eASj7/1s
5+yOpZbtmymSC2tG7IobbbLEtKTuiPQ+DZvevh34on+BWgWEm9B6scgf7rh+jSN1
Qs2zlrPhdYeJ2qWSdp/aMutXPqghyayQfablzbXNnOR/a4CS2qG12ht0n9XBadz1
6rhFKkWaVuvKw+XUgKzz4Lp8B42/GJeTcVluWEg7iNWRBq/9QESpzcz=
-----END CERTIFICATE-----
subject=C = CL, ST = RM, L = Santiago, O = DUOC-UC, OU = VoIP-Lab, CN = 54.80.130.201
issuer=C = CL, ST = RM, L = Santiago, O = DUOC-UC, OU = VoIP-Lab, CN = 54.80.130.201
---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 1513 bytes and written 395 bytes
Verification error: self-signed certificate
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 18 (self-signed certificate)
---
DONE
```

Se utilizó Wireshark con el filtro: `udp.port >= 10000 && udp.port <= 20000`, durante una llamada entre las extensiones 1001 y 1002 se observó tráfico UDP correspondiente al flujo RTP, verificando que RTPengine transporta correctamente el audio.



Se utilizó **Wireshark** con el filtro: `tcp.port == 5061`, se comprobó que la señalización SIP utiliza el protocolo **TLS**, observándose tráfico cifrado entre los clientes MicroSIP y el SBC Kamailio.



Conclusión

El desarrollo de este proyecto permitió implementar una solución de comunicaciones unificadas completa, compuesta por una PBX basada en Asterisk y protegida mediante un Session Border Controller (SBC) Kamailio, ambos desplegados sobre instancias EC2 de AWS. A lo largo del laboratorio se aplicaron conocimientos relacionados con administración de servidores Linux, telefonía IP, redes y seguridad, logrando una plataforma funcional y preparada para operar de forma segura.

En el primer ítem se realizó la instalación y configuración de la PBX Asterisk, creando las extensiones SIP, configurando el servicio PJSIP y el dialplan para permitir la comunicación entre usuarios. Además, se incorporó un script AGI que permitió automatizar parte del flujo de llamadas, comprobando el correcto funcionamiento de la plataforma mediante pruebas entre las extensiones creadas.

En el segundo ítem se implementó un SBC utilizando Kamailio con el objetivo de proteger la infraestructura VoIP. Para ello se configuró el ocultamiento de topología (Topology Hiding), evitando exponer la información interna de la PBX, se integró RTPengine para el manejo del tráfico multimedia y se habilitó la comunicación segura mediante TLS utilizando certificados digitales generados con OpenSSL. Finalmente, se verificó el funcionamiento de todas las configuraciones mediante herramientas como OpenSSL, tcpdump y Wireshark, comprobando el establecimiento del canal TLS, el intercambio seguro de la señalización SIP y el correcto flujo RTP durante las llamadas.

En conclusión, los dos primeros ítems permitieron construir una plataforma VoIP completamente operativa y con mecanismos de seguridad que se utilizan en implementaciones reales. Además de reforzar los conocimientos sobre Asterisk y Kamailio, este proyecto entregó experiencia práctica en el despliegue de servicios sobre AWS, la administración de servidores Linux y la validación de comunicaciones mediante herramientas de análisis de tráfico, fortaleciendo las competencias necesarias para el diseño e implementación de soluciones de comunicaciones unificadas seguras.